Assignment-1 Report

1 Overview

Two different models were built and compared:

- **Gaussian Mixture Model (GMM):** This model that learns what each digit sounds like based on its overall audio shape.
- **Hidden Markov Model (HMM):** This model that tries to understand the order and sequence of sounds in a digit one sound after another.

2 Data and Setup

2.1 Audio Recordings

The recordings are `.wav` audio files, each containing one spoken digit (0 to 9). Files are named like `3_speaker2_40.wav`, which means digit 3 spoken by speaker 2 and 40 represents the number of times speaker 2 said digit 3 (since it starts from 0 , this is the 41st file where 2 said digit 3). The dataset covers 6 different speakers.

Data Split	Speakers Used	Purpose
Training	Speakers 1, 2, 3, 4, 5	Training the models
Validation	Speaker 4	Validating performance on a known speaker
Test	Speaker 6	Testing our trained models

Table 1: Data split

2.2 Feature Extraction

ML models cannot directly understand raw audio waves, so we convert audio into a set of numbers that describe the shape and energy of sound. The feature extraction steps are as follows:

- All audio files are loaded at 16,000 samples per second (16 kHz), which is the standard rate for speech processing.

- **13 MFCCs** (Mel Frequency Cepstral Coefficients) are computed. These are like a fingerprint of the sound at each moment in time, capturing what the voice sounds like rather than the raw waveform.
- **1 Log-Energy** feature is added, capturing how loud each moment is. This helps distinguish between digits.
- **13 Delta** features are computed, measuring how quickly the MFCCs are changing over time.

The final feature for each audio frame is a vector of **27 numbers** (13 MFCC + 1 energy + 13 delta). All features are then normalised to have mean 0 and standard deviation 1 before being fed into either model.

3 Gaussian Mixture Model (GMM)

A Gaussian Mixture Model describes a collection of data points as if they came from several overlapping bell-shaped clusters.

One GMM is trained per digit (10 models total), using only the audio frames from that particular digit. Each GMM uses 4 components (clusters), meaning it learns 4 different sounds that make up each digit. (here 4 is just a hyper parameter I used , it can be tuned to increase the accuracy and other metrics)

3.1 Training (EM Algorithm)

The GMM is trained using Expectation-Maximization (EM), which alternates between two steps until convergence:

- **E-Step:** For each audio frame, calculate how likely it belongs to each of the 4 clusters. These probabilities are called responsibilities.
- **M-Step:** Update the shape and centre of each cluster to better fit the frames assigned to it.

This repeats until the improvement in fit becomes smaller than 10^{-4}. To avoid numerical problems in 27 dimensions, the code uses log-probabilities throughout and adds a regularisation term ($0.01 \times \mathbf{I}$) to each cluster's covariance matrix.

3.2 Prediction

To predict a new spoken digit, its audio frames are scored against all 10 digit GMMs. The digit whose GMM gives the highest average log-likelihood score is declared the predicted digit.

4 Hidden Markov Model (HMM)

A Hidden Markov Model is designed to handle sequences, data that arrives in order over time, like speech. The key idea is that speech passes through a series of hidden states and each state produces certain sounds.

A left-to-right HMM with 3 hidden states per digit is used. (here also 3 is just a hyper parameter) Left-to-right means the model can only move forward through states or stay in the same state, which matches the natural direction of speech.

4.1 Vector Quantisation — Converting Features to Symbols

The HMM (discrete) requires symbolic input, not raw numbers. The continuous audio features are therefore first converted into symbol codes using a **Vector Quantiser (VQ)**:

- A codebook of 64 typical sound patterns (here also 64 is a hyper parameter) is built using k-means clustering on all training frames.
- Each 27-dimensional frame is replaced by the index of the closest codebook entry (an integer from 0 to 63).

A speech sequence originally represented as a stream of 27-number vectors thus becomes a stream of integers such as [3, 3, 41, 41, 7, 19, ...], which the HMM can process.

4.2 Training (Baum-Welch Algorithm)

Each HMM is trained using the Baum-Welch algorithm, the HMM equivalent of EM:

- **Forward Pass:** Compute the probability of reaching each state at each time step.
- **Backward Pass:** Compute the probability of observing the remaining sequence from each state.
- **E-Step:** Combine forward and backward probabilities to estimate how much time was spent in each state and which symbols appeared there.
- **M-Step:** Update transition probabilities and emission probabilities .

4.3 Prediction

To predict a digit, its audio is converted to a symbol sequence via the trained VQ codebook. The sequence is then scored against all 10 HMM models using the forward algorithm, which computes $\log P(\text{obs} \mid \text{model})$. The digit with the highest score is the prediction.

5 Results

5.1 Overall Accuracy Summary

Model	Validation Accuracy (Speaker 4)	Test Accuracy (Speaker 6)
HMM	79.40% (397/500)	76.20% (381/500)
GMM	97.00% (485/500)	84.80% (424/500)

Table 2: Overall accuracy comparison

The GMM outperforms the HMM in both settings. Both models show a drop in accuracy from validation to test, which is expected since the test speaker was never seen during training.

5.2 HMM — Per-Digit Results

5.2.1 Validation (Speaker 4) — Overall Accuracy: 79.40%

Digit	Total Samples	Correctly Predicted	Accuracy
0	50	50	100.00%
1	50	48	96.00%
2	50	38	76.00%
3	50	30	60.00%
4	50	48	96.00%
5	50	30	60.00%
6	50	21	42.00%
7	50	41	82.00%
8	50	41	82.00%
9	50	50	100.00%

Table 3: HMM Validation results

5.2.2 Test (Speaker 6) — Overall Accuracy: 76.20%

Digit	Total Samples	Correctly Predicted	Accuracy
0	50	41	82.00%
1	50	50	100.00%
2	50	31	62.00%
3	50	44	88.00%
4	50	50	100.00%
5	50	48	96.00%
6	50	7	14.00%
7	50	47	94.00%
8	50	47	94.00%
9	50	16	32.00%

Table 4: HMM Test results

5.3 GMM — Per-Digit Results

5.3.1 Validation (Speaker 4) — Overall Accuracy: 97.00%

Digit	Total Samples	Correctly Predicted	Accuracy
0	50	49	98.00%
1	50	50	100.00%
2	50	48	96.00%
3	50	41	82.00%
4	50	49	98.00%
5	50	50	100.00%
6	50	48	96.00%
7	50	50	100.00%
8	50	50	100.00%
9	50	50	100.00%

Table 5: GMM Validation results

5.3.2 Test (Speaker 6) — Overall Accuracy: 84.80%

Digit	Total Samples	Correctly Predicted	Accuracy
0	50	50	100.00%
1	50	50	100.00%
2	50	30	60.00%
3	50	46	92.00%
4	50	50	100.00%
5	50	23	46.00%
6	50	33	66.00%
7	50	47	94.00%
8	50	50	100.00%
9	50	45	90.00%

Table 6: GMM Test results

5.4 Confusion Matrices

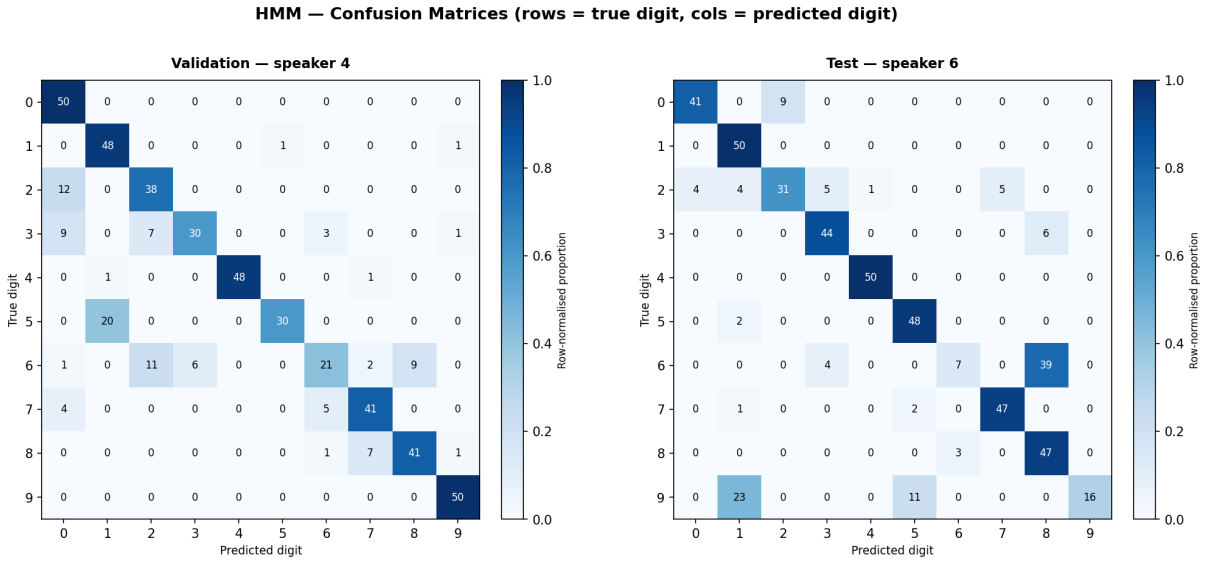

Figure 1: HMM confusion matrices — Validation (left) and Test (right). Rows represent the true digit; columns represent the predicted digit. Colour intensity shows the row-normalised proportion of predictions.

The HMM confusion matrix reveals that the majority of errors are concentrated on digits 2, 3, 5, 6, and 9. On the test speaker, digit 6 almost completely fails (only 7 out of 50 correct), and digit 9 is heavily misclassified as digit 1 (23 out of 50 samples).

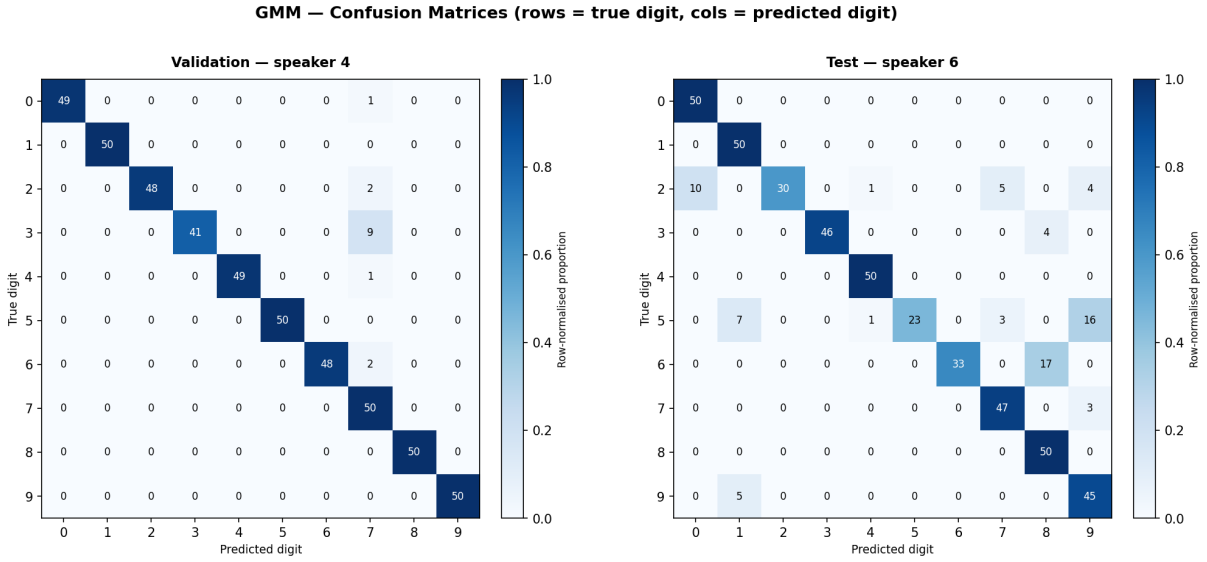

Figure 2: GMM confusion matrices — Validation (left) and Test (right). Rows represent the true digit; columns represent the predicted digit. Colour intensity shows the row-normalised proportion of predictions.

The GMM confusion matrix is considerably cleaner, especially on validation. On the test speaker, the main problem digits are 2, 5, and 6. Digit 5 is frequently confused with 1,

and digit 2 is confused with 0. The diagonal is noticeably stronger and more consistent than in the HMM matrix.

6 Analysis

From the Obtained results we can see that GMM works better than HMM

6.1 Reasons for Better Performance of GMM Than HMM

- **Direct feature modelling.** The GMM works directly with all 27 raw audio features. The HMM first compresses those features into just one of 64 symbols via vector quantisation. Any information lost in that compression cannot be recovered.
- **Quantisation loss.** Even with a codebook of 64 symbols, many different sounds may map to the same symbol. GMM avoids this entirely by modelling the full continuous feature space.
- **Sequence model mismatch.** The HMM is designed in theory to capture the ordering of sounds over time. However, with only 3 states and a limited amount of training data per speaker, the model may not learn a reliable sequence pattern.
- **Simplicity advantage.** A simpler model that uses all available information directly can outperform a more complex model when training data is limited.

6.2 Hardest digits to predict

Certain digits are consistently difficult across both models,

- **Digit 6** is the most problematic. Both HMM and GMM struggle with it on the unseen speaker, with common confusions with digits 2 and 7.
- **Digit 5** is hard for the GMM on the test speaker (46%), frequently confused with digit 1.
- **Digit 9** is hard for the HMM on the test speaker (32%), confused heavily with digit 1.
- **Digit 2** is moderately difficult for both models when facing a new speaker.

By contrast, digits 0, 1, and 4 are consistently easy to recognise, likely because their shapes are distinctive enough to remain separable even across different speakers.

6.3 Overfitting on Validation

The GMM's validation accuracy of 97% is high in part because Speaker 4's data was included in the training set, so the model has already been exposed to that voice. The more realistic measure of generalisation is the 84.8% accuracy on the unseen Speaker 6.

7 Key Design Choices

Several implementation decisions were made to improve reliability and numerical stability:

- **Log-probabilities throughout.** Both models use logarithms of probabilities to prevent numerical underflow — the problem where very small probabilities get rounded to zero, producing incorrect results.
- **Covariance regularisation in GMM.** A small constant ($0.01 \times \mathbf{I}$) is added to each cluster's covariance matrix, preventing it from becoming singular when clusters have very few points assigned.
- **Left-to-right HMM structure.** States can only advance forward or self-loop, matching the natural forward progression of speech.
- **Global feature normalisation.** All features are scaled using statistics computed from the full training set, removing the effect of differences in recording volume or microphone sensitivity etc.

8 Conclusion

In this assignment we implemented two digit recognition models from scratch using numpy without any machine learning libraries. The key findings are :

- The **GMM** achieves 97.00% accuracy on validation and 84.80% on the unseen test speaker, making it the stronger model for this task.
- The **HMM** achieves 79.40% on validation and 76.20% on the test speaker. The information loss from vector quantisation is the primary reason it falls short of the GMM.
- Both models generalise reasonably to a new speaker, though a visible accuracy drop is observed, which is consistent with the challenge of speaker-independent recognition.
- Digits 0, 1, and 4 are reliably recognised by both models. Digits 5, 6, and 9 are the most challenging, particularly for unseen speakers.

Potential improvements include training on more speakers to increase variability coverage, increasing the number of HMM states, using a larger VQ codebook, or replacing the discrete HMM with a continuous-observation HMM to avoid the information loss introduced by vector quantisation(quantisation error).

9 Submission

I have done this assignment using 4 .py files and 2 newly created folders

- **feature_extraction.py**

  In this using librosa, I extracted features for modelling and separated train , test and validation data and saved the test data to test_recordings folder , validation data to val_recordings folder

- **gmm.py**

  This file has implementation of gmm model

- **hmm.py**

  This file has implementation of hmm model

- **predict.py**

  This file predicts the both models and gives accuracy, per digit performances and confusion matrices using both test and validation data

All the required files, folders are present in the .zip itself , to check results just directly download and run the .py files